

Artificial Intelligence and Applications (BTIT13602)

Unit-6: Prolog Programming

drishi

- **Dr. Mitali H. Desai**
Assistant Professor,
Information Technology Department

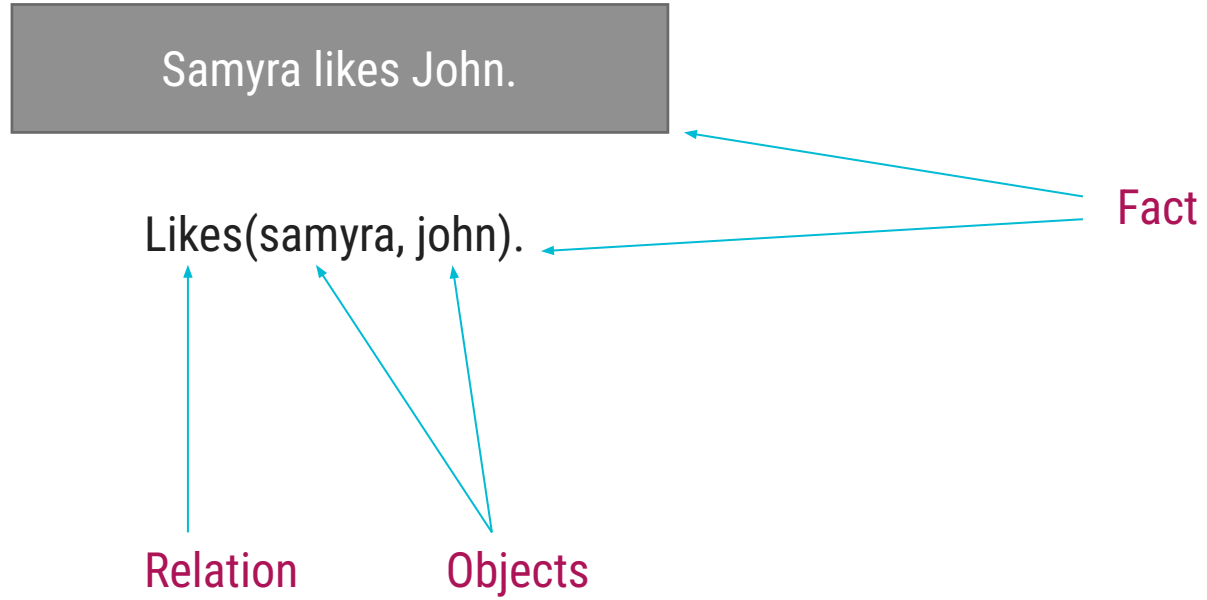
Outline

- Introduction
- Facts
- Objects
- Relations
- Predicates
- Structure of Prolog Program
 - Examples
- Recursion in Prolog
- Cut and fail
- List
 - List Examples

Introduction

- Prolog stands for **Programing in logic**.
- Prolog is a **declarative programming language** unlike most common programming languages.
- In a declarative language the programmer specifies a goal to be achieved and then the Prolog system works out how to achieve it.
 - Prolog is a declarative language, which means that a program consists of data based on the facts and rules, i.e., relationships among these facts.
- Rather than describing how to compute a solution, a program consists of a database of facts and logical relationships (rules).
- The user asks a question to obtain a solution of a problem.
 - When asked a question, the run time system searches through the database of facts and rules to determine (by logical deduction) the answer.
- Prolog is used some areas like natural language processing, artificial intelligence, expert systems, automated reasoning, etc.

Facts, Objects, Relations



Facts, Objects, Relations

- Facts can be described as a symbolic Relationships
- Facts are statements about what is true about a problem, instead of instructions how to accomplish the solution.
- The Prolog system uses the facts to work out how to accomplish the solution by searching through the space of possible solutions.
- Example:-
 - Bob is a student.
 - student(bob).
- This expression is called a **clause**.
- An **object** is the name of element of certain type.
- A **Relation** is a name that defines the way in which a collection of objects or variables belong together.

Predicate

- A **relation identifier** is referred to as a predicate
- Example:-
 - Bob is a student.
 - student(bob).
- Predicate express a **relationship**.
- The element within the parenthesis are the arguments of the predicate, which may be objects or variable.
- The word before the parenthesis is the name of **relation**.

Rules

- Rules are used when you want to say that a fact depends on a group of facts.
- Rule consist of a head and a body connected by the symbol :- (IF)
- **Syntax of rule**
 - Read ‘:-’ as ‘if’.
 - *likes(john,X) :- likes(X,cricket).*
 - “John likes X if X likes cricket”.
 - Rules always end with ‘.’

Structure of Prolog Program

Domains

```
// defining Objects, variable  
//Person1,Person2 = Symbol
```

Predicates

```
// defining a Relation and Function  
//likes(Person1,Person2)
```

clauses

```
// defining Facts, Rules  
// likes(aditya, naina).  
//likes(bob, hezal).
```


Structure of Prolog Program

- **Domain Section** : Programmer can define the type of each object

char : single character (enclosed in ‘ ’)

integer : integer from -32768 to 32767

real : floating point number

string : character sequence (enclosed in “ ”)

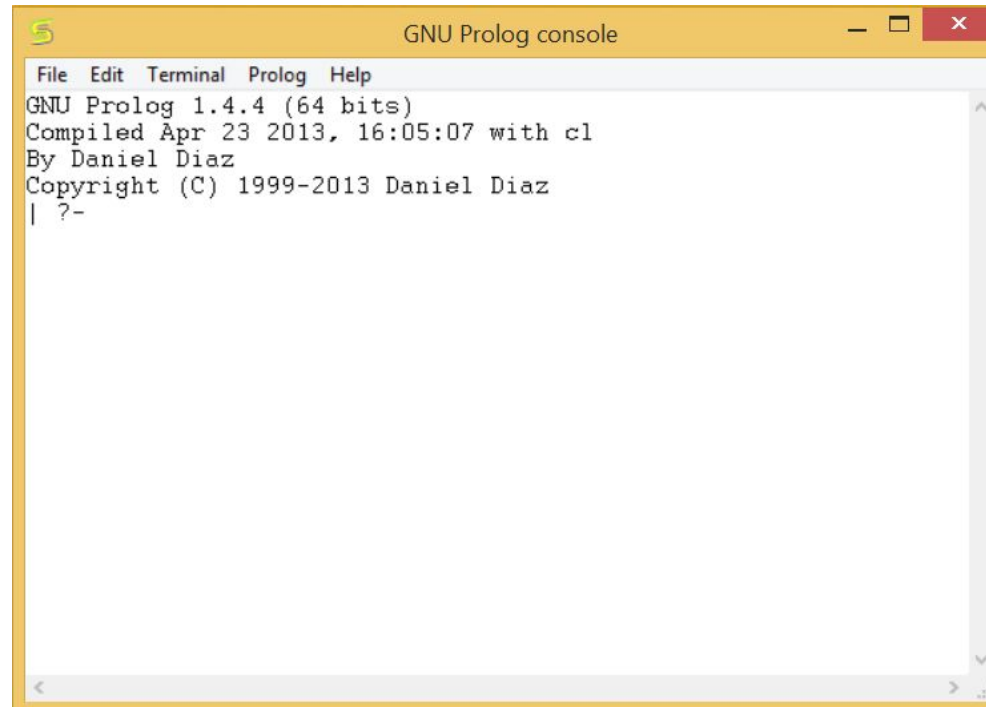
symbol : character sequence of letters, numbers and underscores with the first character a lowercase letter

file : symbolic file name

- **Predicates Section** : Predicates (relations) used in the clauses section are defined. Each relation used in the clauses of the clauses section must have a corresponding predicate definition in the predicates section. Except for the built in predicates of Turbo Prolog.
- **Clauses Section** : The main body of the prolog program. Contains the clauses that define the program – facts and rules.

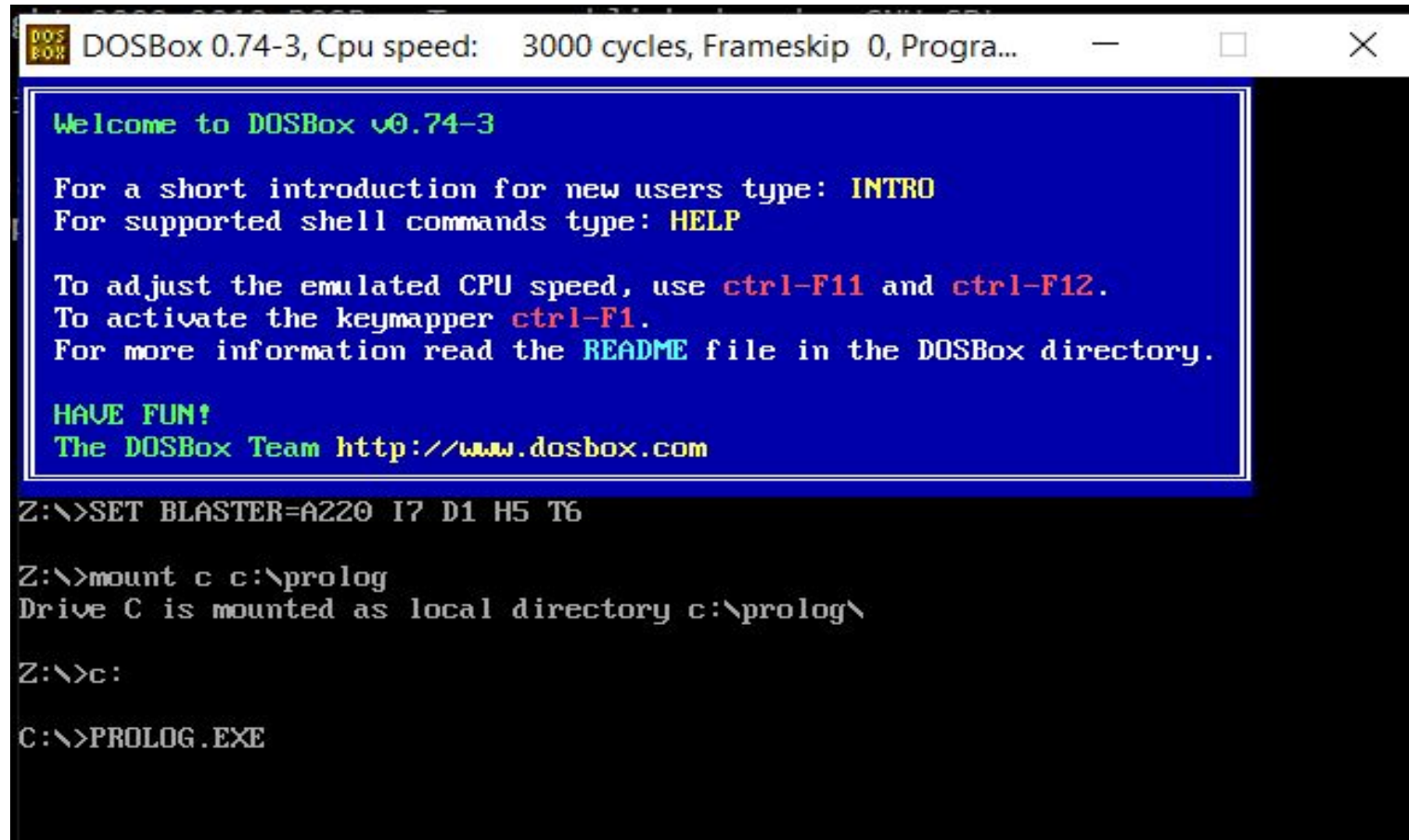
Installing GNU prolog Compiler

- This is a prolog interpreter working under the GNU licence. Nice things about it are:
Windows/UNIX support, comprehensive, free.
 - Download GUN prolog form <http://www.gprolog.org/>
 - Double click setup file to install
 - After installation complete, run GUN Prolog



Installing DOSBOX

- Prolog Programming- DOSBOX turbo prolog



The screenshot shows a DOSBox window titled "DOSBox 0.74-3, Cpu speed: 3000 cycles, Frameskip 0, Progra...". The window has a blue title bar and standard Windows window controls. The main area is black with green text. A white-bordered box contains the welcome message. Below this, the user enters commands at the Z:\> prompt, and the prompt changes to C:\> after mounting the drive.

```
DOSBox 0.74-3, Cpu speed: 3000 cycles, Frameskip 0, Progra...
Welcome to DOSBox v0.74-3
For a short introduction for new users type: INTRO
For supported shell commands type: HELP

To adjust the emulated CPU speed, use ctrl-F11 and ctrl-F12.
To activate the keymapper ctrl-F1.
For more information read the README file in the DOSBox directory.

HAVE FUN!
The DOSBox Team http://www.dosbox.com

Z:\>SET BLASTER=A220 I7 D1 H5 T6

Z:\>mount c c:\prolog
Drive C is mounted as local directory c:\prolog\

Z:\>c:

C:\>PROLOG.EXE
```

Prolog Programming- DOSBOX turbo prolog

Here's a basic example of Prolog that you might use on an older DOS-based Prolog interpreter like Turbo prolog, SWI-Prolog or other similar systems.

Let's define a simple knowledge base and some rules. For example, we'll create a small database of family relationships.

Example: Family Relationships

Facts:

1. `parent(alice, bob).` means Alice is a parent of Bob.
2. `parent(bob, carol).` means Bob is a parent of Carol.
3. `parent(carol, dave).` means Carol is a parent of Dave.

Rules:

- `grandparent(X, Y) :- parent(X, Z), parent(Z, Y).` means X is a grandparent of Y if X is a parent of Z and Z is a parent of Y

Example

The screenshot shows a DOSBox window titled "DOSBox 0.74-3, Cpu speed: 3000 cycles, Frameskip 0, Progra...". The main window is a Prolog editor with a blue background and yellow text. The editor has a menu bar with "Files", "Edit", "Run", "Compile", "Options", and "Setup". Below the menu bar is a status bar showing "Line 5", "Col 23", "C:\RELATION.PRO", "Indent", and "Insert". The editor contains the following code:

```
Domains
person = symbol

Predicates
parent(person, person)
grandparent(person, person)

Clauses
parent(alice, bob).
parent(bob, carol).
parent(carol, dave).
grandparent(X, Y) :-
    parent(X, Z),
    parent(Z, Y).
```

On the right side of the editor, there are two panels. The top panel is titled "Dialog" and shows the following text:

```
Goal: parent(alice,bob)
Yes
Goal: grandparent(X,dave)
X=bob
1 Solution
Goal: grandparent(alice,bob)
No
Goal: _
```

The bottom panel is titled "Message" and shows the following text:

```
Save C:\RELATION
.PRO
Compiling C:\REL
ATION.PRO
parent
grandparent
parent
```

At the bottom of the window, there is a status bar with the following text: "F2-Save F3-Load F5-Zoom F6-Next F8-Previous goal Shift-F10-Resize F10-End".

Example

The screenshot shows the DOSBox 0.74-3 Prolog editor window. The title bar reads "DOSBox 0.74-3, Cpu speed: 3000 cycles, Frameskip 0, Progra...". The menu bar includes "Files", "Edit", "Run", "Compile", "Options", and "Setup". The main editor area has a status bar at the top showing "Line 1", "Col 1", "C:\SUM.PRO", "Indent", and "Insert". The editor content is as follows:

```
Domains
    Person1, Person2 = Symbol
Predicates
    likes(Person1, Person2)
Clauses
    likes(aditya, naina).
    likes(bob, hezal).
```

On the right side, there are two panels. The "Dialog" panel shows the current goal state:

```
Goal: likes(bob,
hezal)
No
Goal: likes(bob,
hezal)
Yes
Goal:
```

The "Message" panel shows the execution progress:

```
Load WORK.PRO
Load C:\SUM.PRO
Compiling C:\SUM
.PRO
likes
```

At the bottom, a status bar displays keyboard shortcuts: "F2-Save F3-Load F5-Zoom F6-Next F8-Previous goal Shift-F10-Resize F10-End".

Example

DOSBox 0.74-3, Cpu speed: 3000 cycles, Frameskip 0, Program: PROLOG

Files Edit Run Compile Options Setup

Editor

Line 4 Col 1 C:\MAX.PRO Indent Insert

```
Domains
value = Integer

Predicates
max(value,value,value)

Clauses
max(X,Y,X) :-
    X>Y.
max(X,Y,Y) :-
    Y>X.
```

Dialog

Goal: max(3,4,MAX)
MAX=4
1 Solution
Goal: max(5,2,X)
X=5
1 Solution
Goal: _

Message

Compiling WORK.PRO
Save C:\MAX.PRO
Load C:\MAX.PRO
Compiling C:\MAX.PRO
max

Trace

F2-Save F3-Load F5-Zoom F6-Next F8-Previous goal Shift-F10-Resize F10-End

List

- It is a finite **sequence of elements**.
- A list in PROLOG is a structure of the form
 $[t_1, t_2, t_3 \dots t_n]$
- The elements in list organised in two section **head** and **tail**.
- The direct access to only one element call **head** , while the rest forms the list called the **Tail**.
 $[Head|Tail]$
 - where Head is a single element, while Tail is list.

List

[] is the empty list.

- [x, 2+2, [a, b, c]] is a list of three elements.
- The first element in the list is its **head**.
- The list with the head removed is the **tail**.
 - The head of [x, 2+2, [a, b, c]] is x
 - The tail is [2+2, [a, b, c]]

List

- [] % empty list
- [a] % singleton list
- [hello, world] % 2 element list
- [[1,2,3,4], p, this] % 3 element list
- [[[1, 2], [3, 4]], [[a, b], [x, y]]]. % nested list (3 level nesting)

List

- $[a \mid L]$
- Here L is a list and a is a single element.
- If, $L = [b, c, d]$
- Then, $[a \mid L]$ will result in $[a, b, c, d]$

List

- Unification can be performed on lists:
 - $[a, b, c] = [X, Y, Z]$
results in $X = a, Y = b, Z = c$
 - $[a, b, c] = [\text{Head} \mid \text{Tail}]$
results in $\text{Head} = a, \text{Tail} = [b, c]$
- Nonempty lists can be matched against $[\text{Head} \mid \text{Tail}]$.
- Empty lists will not match $[\text{Head} \mid \text{Tail}]$.
- The tail of a list is always itself a list.
- $[X \mid Y, Z]$ isn't legal.

Making Use of Unification

- $[a,b,c]$ unifies with $[Head|Tail]$ resulting in $Head=a$ and $Tail=[b,c]$
- $[a]$ unifies with $[H|T]$ resulting in $H=a$ and $T=[]$
- $[a,b,c]$ unifies with $[a|T]$ resulting in $T=[b,c]$
- $[a,b,c]$ doesn't unify with $[b|T]$
- $[]$ doesn't unify with $[H|T]$
- $[]$ unifies with $[]$. Two empty lists always match

■ Member of a list

domains

integerlist = integer*

predicates

member(integer, integerlist)

Clauses

member(X, [X|_]).

member(X, [_|Tail]) :- member(X, Tail).

DOSBox 0.74-3, Cpu speed: 3000 cycles, Frameskip 0, Program: PROLOG

Files Edit Run Compile Options Setup

Editor

Line 1 Col 1 C:\LIST.PRO Indent Insert

```
domains
integerlist = integer*

predicates
    member(integer, integerlist)

Clauses
    member(X, [X|_]).
    member(X, [_|Tail]) :- member(X, Tail).
```

Dialog

Goal: member(4, [1,3,4,5])
Yes
Goal: member(4, [1,5,6])
No
Goal: _

Message

Load WORK.PRO
Load C:\LIST.PRO
Compiling C:\LIST.PRO
member

Trace

F2-Save F3-Load F5-Zoom F6-Next F8-Previous goal Shift-F10-Resize F10-End

■ Find length of list

Domains

sybollist = symbol*

integerlist = integer*

predicates

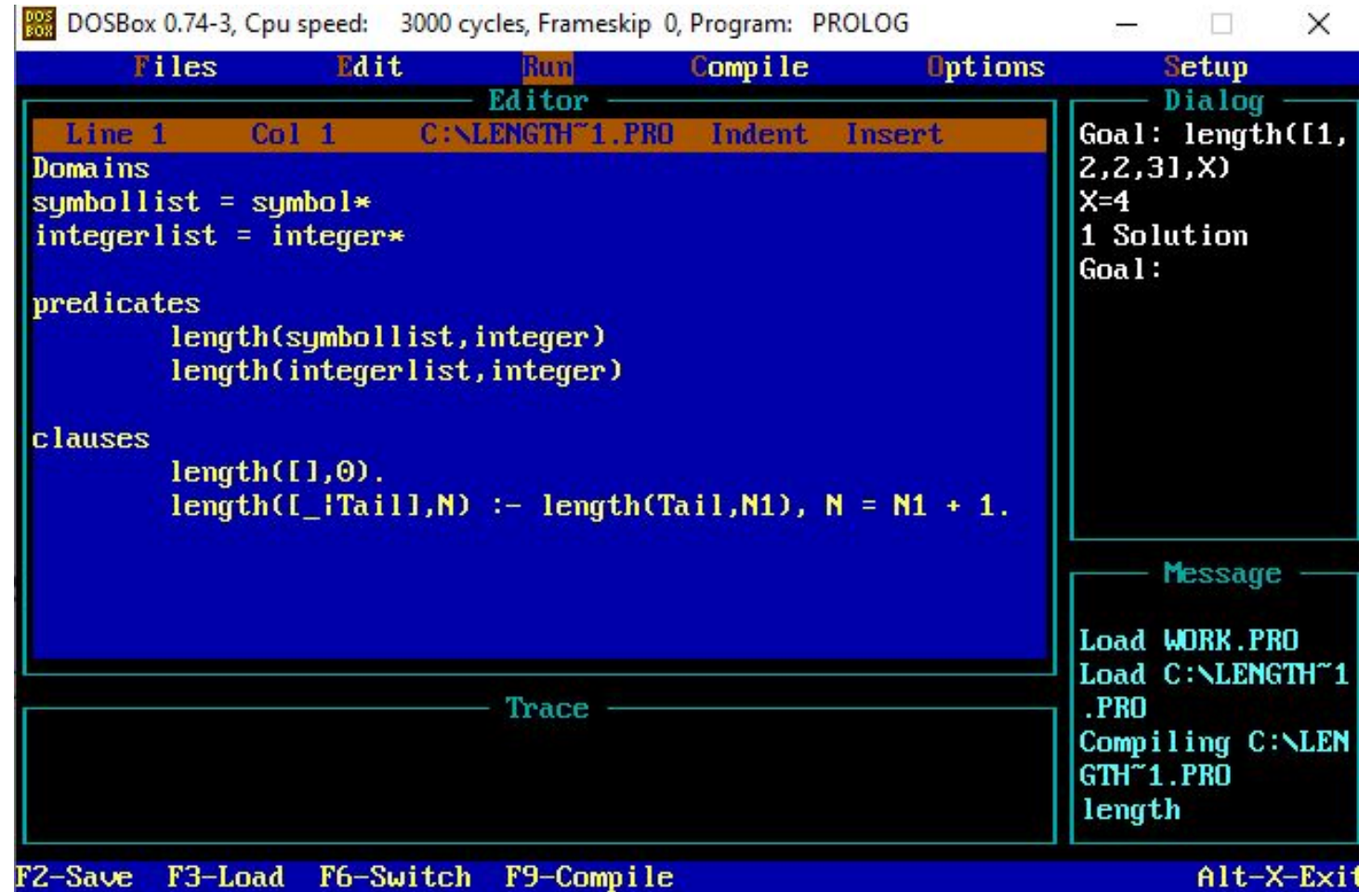
length(sybollist,integer)

length(integerlist,integer)

clauses

length([],0).

length([_|Tail],N) :- length(Tail,N1), N = N1 + 1.



```
DOSBox 0.74-3, Cpu speed: 3000 cycles, Frameskip 0, Program: PROLOG
Files Edit Run Compile Options Setup
Line 1 Col 1 C:\LENGTH~1.PRO Indent Insert
Domains
sybollist = symbol*
integerlist = integer*
predicates
length(sybollist,integer)
length(integerlist,integer)
clauses
length([],0).
length([_|Tail],N) :- length(Tail,N1), N = N1 + 1.
Goal: length([1, 2, 2, 3],X)
X=4
1 Solution
Goal:
Load WORK.PRO
Load C:\LENGTH~1.PRO
Compiling C:\LENGTH~1.PRO
length
F2-Save F3-Load F6-Switch F9-Compile Alt-X-Exit
```

■ Concatenation of two lists

domains

list = symbol*

predicates

appendlist(list,list,list)

clauses

appendlist([],List,List).

appendlist([X|List1] , List2 , [X|List3]) :- appendlist(List1,List2,List3).

■ Delete an item from a list

domains

list=symbol*

predicates

delete(symbol,list,list)

clauses

delete(X,[X|L1],L1).

delete(X,[Y|L1],[Y|L2]) :- delete(X,L1,L2).

Tracing a Prolog Program

- Write trace
- Keep pressing F10

DOSBox 0.74-3, Cpu speed: 3000 cycles, Frameskip 0, Program: PROLOG

Files Edit Run Compile Options Setup

Editor

Line 1 Col 1 C:\LENGTH~1.PRO Indent Insert

```
trace
Domains
symbolist = symbol*
integerlist = integer*

predicates
    length(symbolist, integer)
    length(integerlist, integer)

clauses
    length([], 0).
    length([_!Tail], N) :- length(Tail, N1), N = N1 + 1.
```

Dialog

Goal: length([1, 2, 2, 3], X)
X=4
1 Solution
Goal: length([1, 2, 3, 4], X)
X=4
1 Solution
Goal:

Message

.PRO
Compiling C:\LEN
GTH~1.PRO
length
Compiling C:\LEN
GTH~1.PRO
length

Trace

4=4
RETURN: length([1, 2, 3, 4], 4)

F1-Help F2-Save F3-Load F5-Zoom F6-Next F7-Xcopy F8-Xedit F9-Compile F10-Menu

Recursion in prolog

- Any function which calls itself is called **recursive function**.
- In Prolog, recursion appears when a predicate contain a goal that refers to **itself**.
- This simply means a program calls itself typically until some **final point is reached**.
- In Prolog and in any language, a recursive definition always has at **least two parts**.
- A first fact that act like **a stopping condition** and a rule that call itself simplified.
- At each level the first fact is checked. If the fact is true then the **recursion ends**. If not the recursion **continue**.
- A recursive rule must never call itself with the same **arguments**. If that happens then the program will never end.

Find factorial of given number

fact.pro

```
fact(0,Result) :-  
    Result is 1.
```

```
fact(N,Result) :-  
    N > 0,  
    N1 is N-1,  
    fact(N1,Result1),  
    Result is Result1*N.
```

Output

```
Goal:fact(2,F).  
F = 2  
Goal:fact(3,F).  
F = 6
```

The image shows the SWISH Prolog IDE interface. The top menu bar includes 'File', 'Edit', 'Examples', and 'Help'. Below the menu, there are tabs for 'examples' and 'Program'. The 'Program' tab is active, displaying the Prolog code for calculating the factorial of a number. The code is as follows:

```
1 % Base case: Factorial of 0 is 1  
2 factorial(0, 1).  
3  
4 % Recursive case: Factorial of N is N * factorial(N-1)  
5 factorial(N, Result) :-  
6     N > 0,  
7     N1 is N - 1,  
8     factorial(N1, Result1),  
9     Result is N * Result1.  
10
```

Below the code editor, there is a console window titled 'factorial(5,X)'. It shows the result 'X = 120' and buttons for 'Next', '10', '100', '1,000', and 'Stop'. The console also displays the query '?- factorial(5,X)'.

Find the sum of first N natural numbers

Number.pro

```
sum(0,0).  
sum(N,R):-  
    N > 0,  
    N1 is N-1,  
    sum(N1,R1),  
    R is R1+N.
```

Output

```
Goal: sum(4,SUM).  
SUM = 10
```

Fail

- Fail predicate simply **fails the rule**.
- A typical use of **fail** is a negation of a predicate
- When Prolog fails, it tries to **backtrack**. Thus fail can be viewed as an instruction to force **backtracking**.

Fail.pro

```
a(X) :- b(X), c(X), fail.
```

```
a(X) :- d(X).
```

```
b(1).
```

```
b(4).
```

```
c(1).
```

```
c(3).
```

```
d(4).
```

- **a(1)** will fail because the first clause forces failure after checking **b(1)** and **c(1)**.
- **a(4)** will succeed because the second clause is satisfied by **d(4)**.

So, after running this Prolog program, the only solution will be **a(4)**.

Output

```
Goal: a(x).
```

```
X = 4
```

Fail

The **fail** predicate is used to force the failure of a goal. When **fail** is encountered, Prolog immediately backtracks and attempts to find an alternative solution (if possible).

- **Purpose:**

- It can be used to explicitly fail a goal and trigger backtracking.
- Often used in combination with the cut to control the flow of the program.

Cut

- Sometimes it is desirable to selectively turn off **backtracking**.
- Cut always **succeeds**, but cannot be **backtracked**.
- The cut effectively tells Prolog to freeze all the decisions made so far in this **predicate**.
- That is, if required to backtrack, it will automatically fail without trying other alternatives.
- Performance is the main reason to use the cut.
- The Symbol of cut predicate is **“!”**

cut.pro

```
a(X) :- b(X),!,c(X).  
a(X) :- d(X).
```

```
b(1).  
b(4).  
c(1).  
c(3).  
d(4).
```

Output

```
Goal:a(x).  
X = 1
```

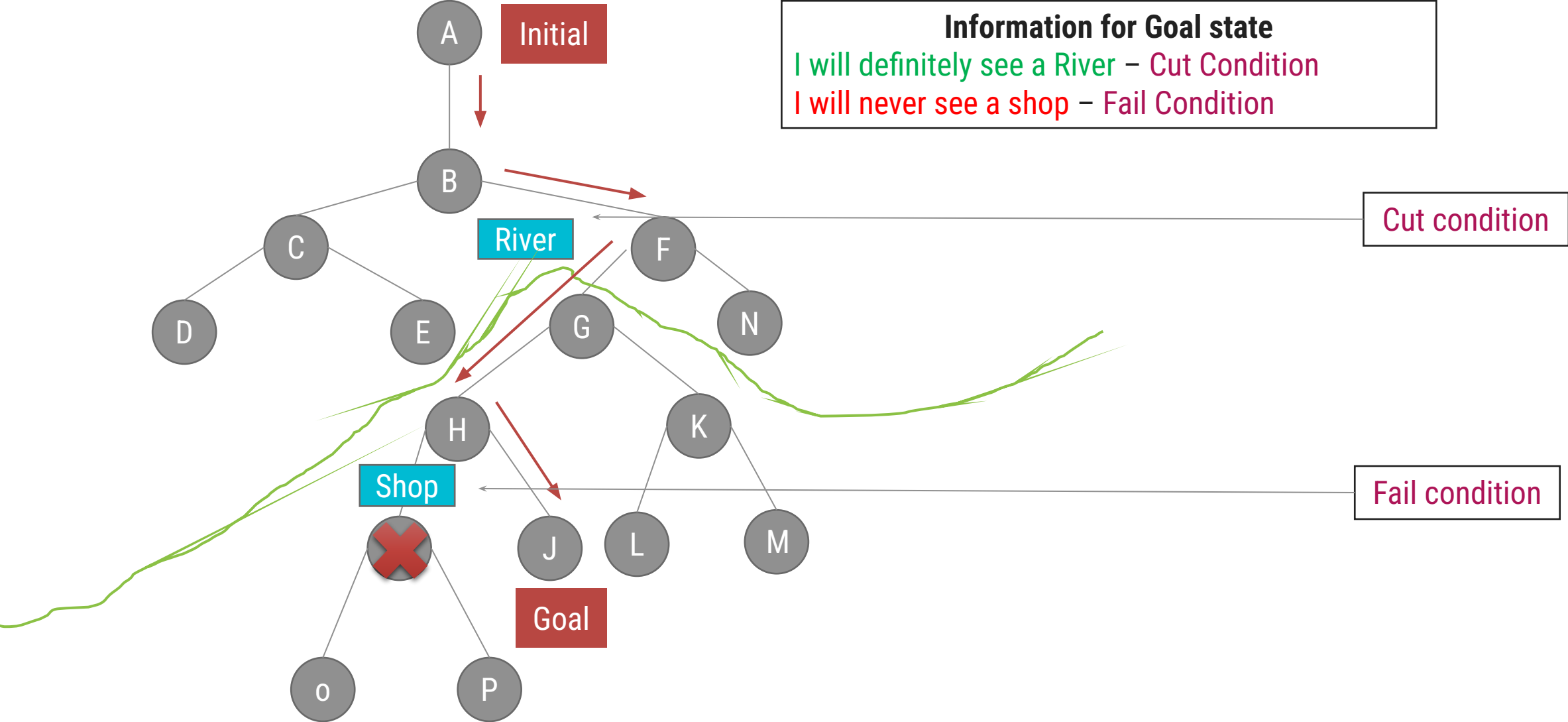

Cut

The **cut** (!) is used to prevent further backtracking once a certain condition is satisfied. When Prolog encounters a cut, it commits to the choices made up to that point, meaning it will not consider alternative solutions after the cut.

- **Purpose:**

- To stop Prolog from exploring alternative clauses after a match.
- It can be used for optimization or to prevent unwanted backtracking.

Cut and Fail With Example



Applications of Prolog

- Expert Systems
- Natural Language Processing
- Robotics
- Gaming
- Simulation

Features of Turbo Prolog

Advantages of using Prolog :

- It is a true compiler. It can generate stand-alone programs that can execute on a machine not running Turbo Prolog.
- Standard predicates for many functions such as string operations, random file access, cursor control, graphics, windows and sound is available.
- Procedural language support can be added to a Prolog system through functional interface
- Both integer and real arithmetic is supported.
- An integrated editor is provided, so that program development, debugging and compilation is easy

Limitations of Prolog :

- Not very efficient for numerical processing
- All data structures used in conventional procedural programming are not available with Prolog

Thank You!